Applied Gen AI Research

RAG & Agent Development - full syllabus

Format: Live, 5 sessions x 2 hours = 10 hours
Cadence: A new cohort starts every 45 days
Seats: 16 maximum
Level: Highly technical - you write code during the sessions
Price: Rs. 24,999 (list Rs. 50,000)

This cohort is for people who already write code and are tired of tutorials that stop at the first working demo. We read four foundational papers properly - method, ablations, and what actually replicates - and build from them. By the end you will have assembled a retrieval pipeline yourself rather than imported one, and an agent loop you can reason about when it misbehaves.

Prerequisites

- Comfortable writing Python day to day.
- You have called an LLM API at least once.
- Basic familiarity with vectors and embeddings helps but is not required.
- A machine you can run Python on, and an API key for at least one model provider. We will tell you which before session one.

The four papers

- Lewis et al. (2020) - Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401
- Asai et al. (2023) - Self-RAG: Learning to Retrieve, Generate and Critique through Self-Reflection. arXiv:2310.11511
- Yao et al. (2022) - ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629
- Schick et al. (2023) - Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761

Session by session

Session 1 - Retrieval, from first principles

Paper: Lewis et al. (2020)

- Why parametric knowledge fails, and precisely which failures retrieval fixes.
- The architecture the original RAG paper proposed, and how far current practice has drifted from

it.

- Set up the codebase; index a real corpus; get a naive pipeline answering questions.
- Between sessions: index a corpus of your own and bring three answers it gets wrong.

Session 2 - Building the RAG framework

Paper: Asai et al. (2023)

- Chunking strategies and what each one costs you at retrieval time.
- Embedding model choice: when it matters, when it is noise.
- Hybrid dense and sparse search; why BM25 refuses to die.
- Reranking, and where it earns its latency.
- Self-reflection as a retrieval control loop - what Self-RAG adds and what it costs.

Session 3 - Evaluation and failure analysis

Paper: No new paper - we work on your pipelines

- Retrieval metrics that predict downstream quality, and the ones that do not.
- Groundedness and citation checking without a human in the loop.
- Building a regression test set from your own failures.
- Separating retrieval failures from generation failures - the single most useful diagnostic skill in this field.

Session 4 - Agents: the reasoning-acting loop

Paper: Yao et al. (2022)

- The ReAct pattern, and what it actually claims.
- Tool calling: schema design, error surfaces, and what the model does with a failed call.
- Structuring an agent loop you can debug - state, budget, termination.
- Convert your RAG pipeline into a tool the agent calls.

Session 5 - Tool use, memory and production hardening

Paper: Schick et al. (2023)

- Self-supervised tool learning and what it implies for how you design tools.
- Memory: what to persist, what to recompute, and the cost of getting it wrong.
- Loop control, budget caps and graceful degradation.
- The failure modes that only appear under real traffic, and how to instrument for them.

What you leave with

- A retrieval pipeline you built, not a template you cloned.
- An agent loop with tool calling, memory and failure recovery.
- An evaluation harness that catches regressions before your users do.

- The ability to read the next paper in this field without waiting for a summary.
